



编译原理

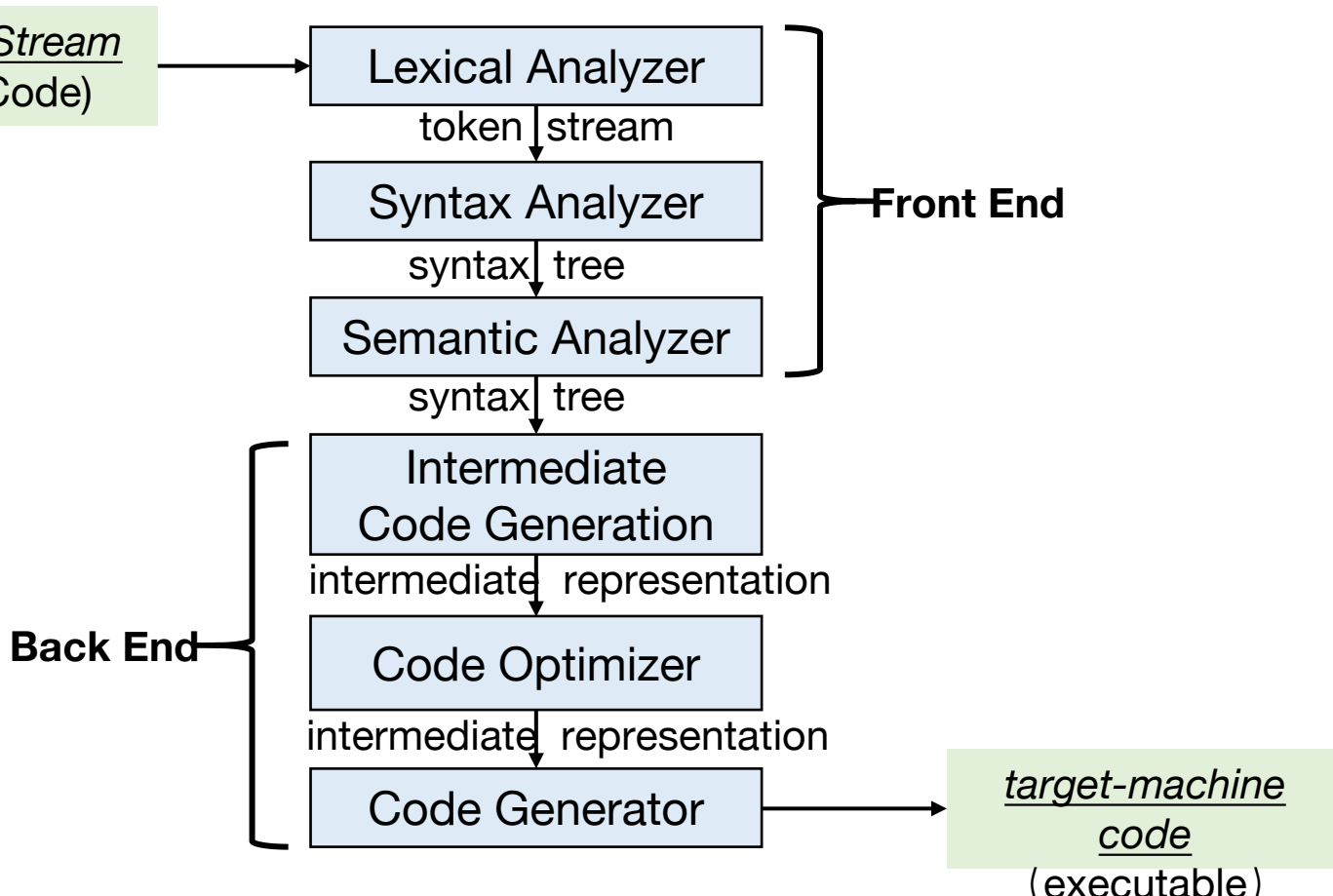
Compiler Principles

Final Lecture

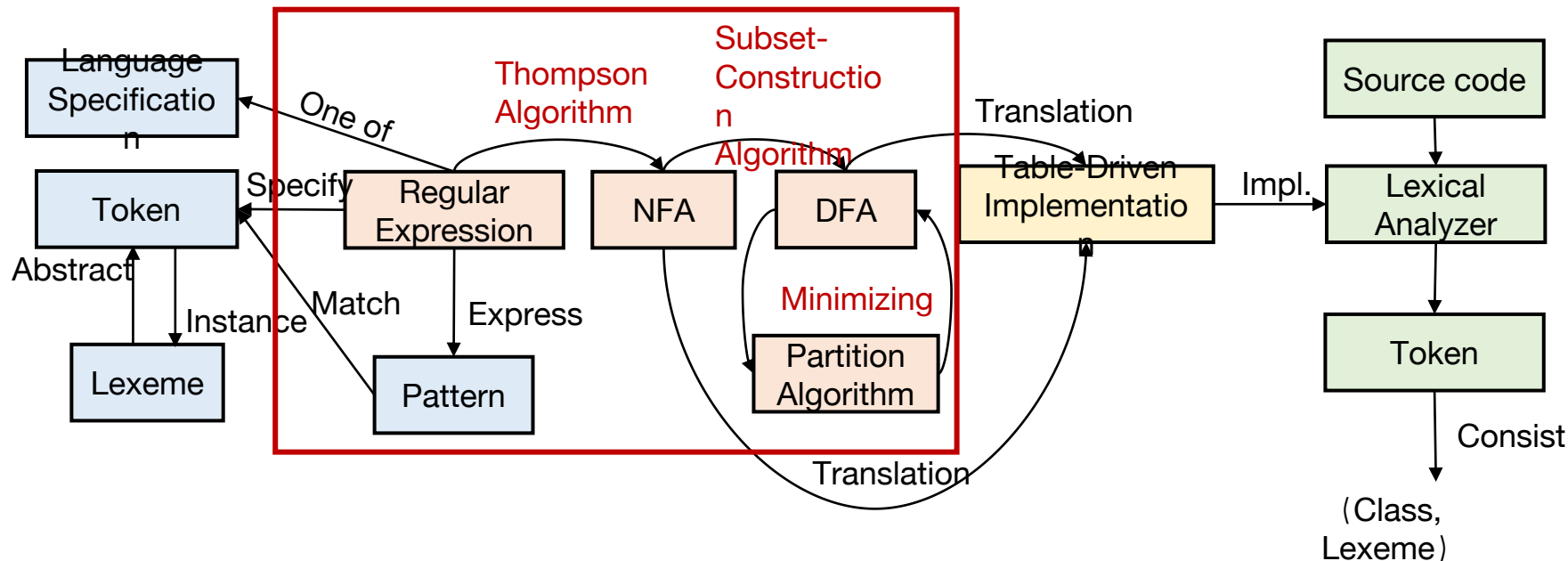
A Review of Compiler Principles

赵帅
计算机学院
中山大学

Compilation Phases[编译阶段]



Final Review—Lexical Analysis



Final Review—Lexical Analysis



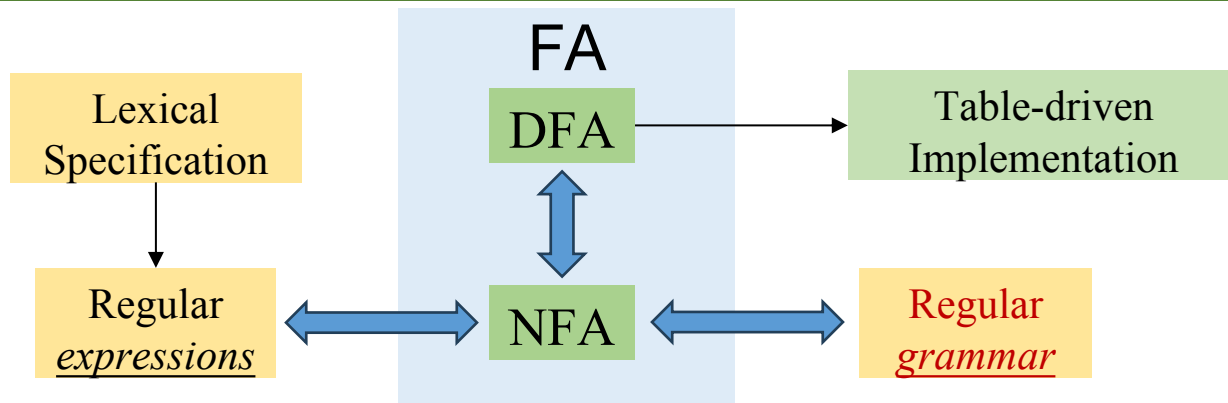
- A **token**[词法单元] is a tuple.
 - (token name, optional attribute value(**lexeme**))
- **Alphabet** Σ [字母表]; **String**[串]: 字母表中符号的有穷序列。
- **Language**[语言]: 某个给定字母表上一个任意的可数的串集合。
- How to describe string patterns? [如何去描述上面的集合]
 - **Regular Expressions**. [使用RE去描述正则语言/正规集]
- Atomic[原子] Regular Expressions, $(r)|(s)$, $(r)(s)$, $(r)^*$...
 - ? 通过RE计算表示的语言对应语言上的运算: **Union**[并], **Concatenation**[连接], **Closure**[闭包]
- ? Description of Language to Regular Expressions. Another form is not to include a certain substring **aba**.
 - All strings of a's and b's which contains the substring **aba**.
 - $(a|b)^*aba(a|b)^*$, 分段、枚举...

```
while (y<z){  
    int x = a +  
    b;  
    y += x; }
```

token

(keyword,	(id, S)
while)	(sym, ;)
(id, y)	(id, y)
(sym, <)	(sym, +=)
(id, z)	(id, x)
(id, x)	(sym, ;)
(id, a)	
(sym, +)	

Final Review—Lexical Analysis



NFA, DFA, How to determine a FA is DFA or NFA ?

Transition Flow:

- 1 ? Converting REs to NFA: Thompson Algorithm [必须使用汤普森构造法]
- 2 ? Converting NFA to DFA: Subset-Construction Algorithm [子集构造法]
- 3 ? Minimizing DFA: Partition Algorithm [分割法]

[Dragon Book 2nd edition, sec 3.9.6, Algorithm 3.39]



Final Review—Lexical Analysis



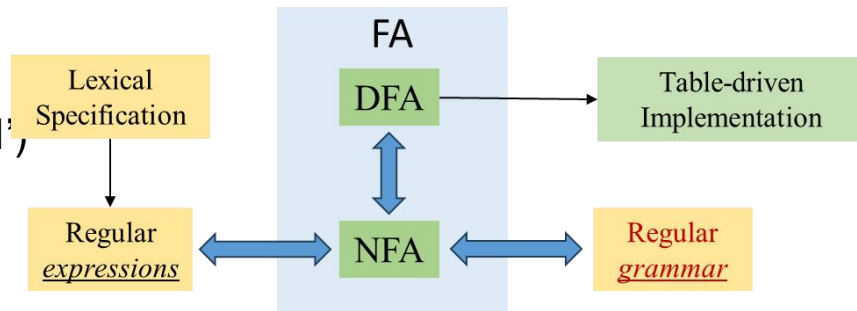
一些等价性的结论

DFA与NFA

- 对于每个NFA M 存在一个DFA M' , 使得 $L(M)=L(M')$;
 - DFA与NFA描述能力相同!

正则表达式与FA

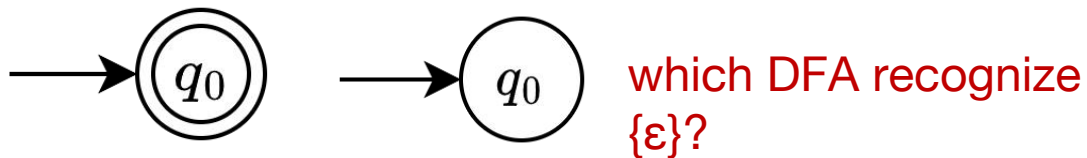
- 对任何FA M , 都存在一个正规式 r , 使得 $L(r)=L(M)$ 。
- 对任何正规式 r , 都存在一个FA M , 使得 $L(M)=L(r)$ 。



Final Review—Lexical Analysis



- 确定有限自动机(DFA)中不存在 ϵ 弧, 所以DFA无法识别空串 ϵ 吗?



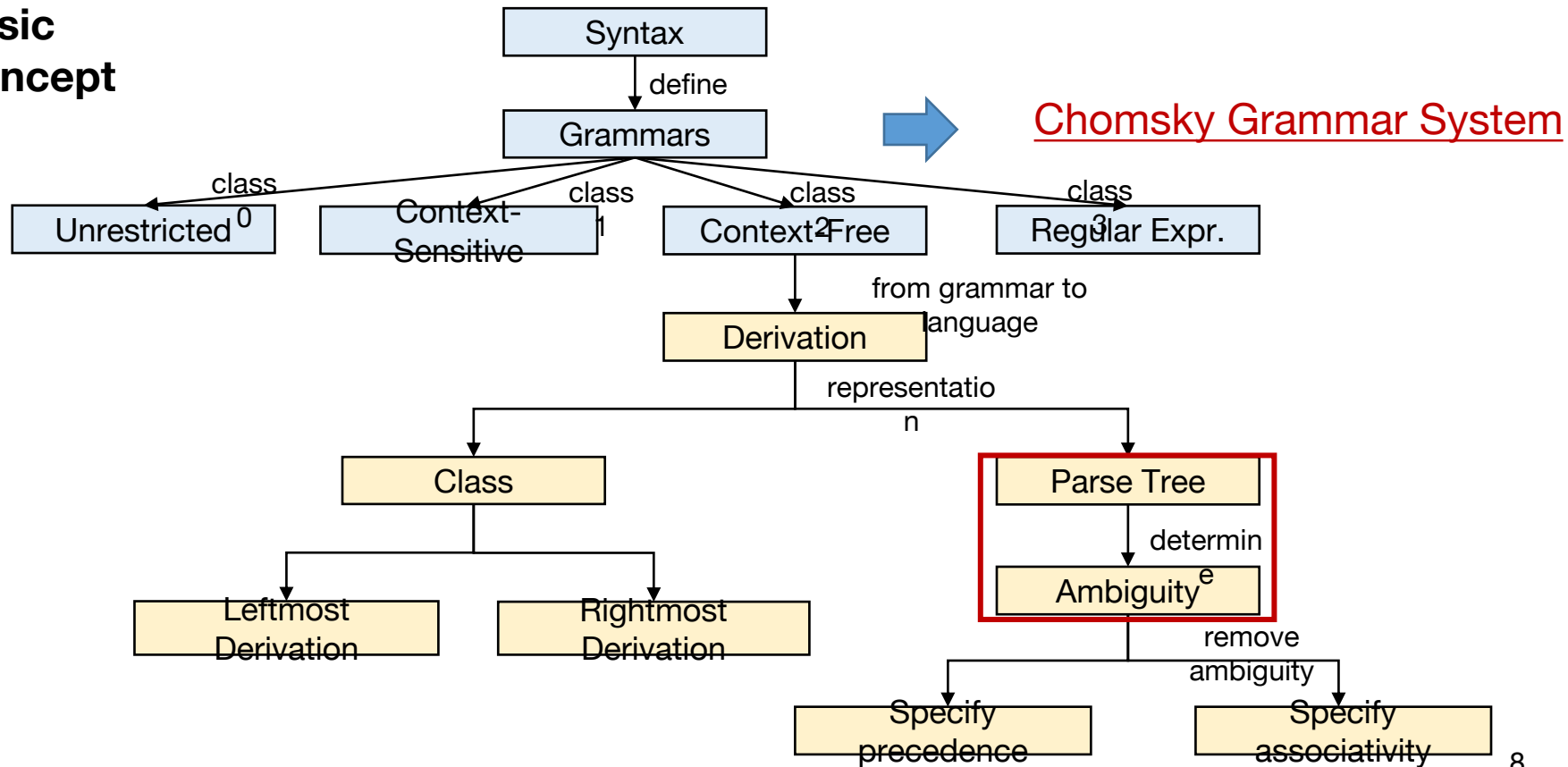
- 正则表达式可以表达语言 $L = \{0^n 2 1^n \mid 1 \leq n \leq 10\}$ 吗?
- Remember, to show a language is regular, we gave a DFA.
- 正则表达式可以表达语言 $L = \{0^n 2 1^n \mid 1 \leq n\}$ 吗?



Final Review—Syntax Analysis



Basic Concept



Final Review—Syntax Analysis



- **Grammar** is to describe the syntax of programming language.

δ : LHS $\rightarrow$ RHS “定”

- Formal definition : 4 components $G=(V_T, V_N, S, \delta)$

- Grammar Convention[约定]

- $? V_T, ? V_N$, 文法符号串, 规则合并, $? G/G(S)/G[S]$

- **? Chomsky Grammar System**

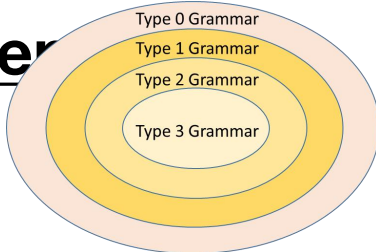
- with different restrictions on δ .

- Type 0,1,2,3.

- **? Determine the most precise type of grammar.**

$G: E \rightarrow E + T \mid E - T \mid T$
 $T \rightarrow T * F \mid T / F \mid F$
 $F \rightarrow (E) \mid id$

Type 2 or Context-Free

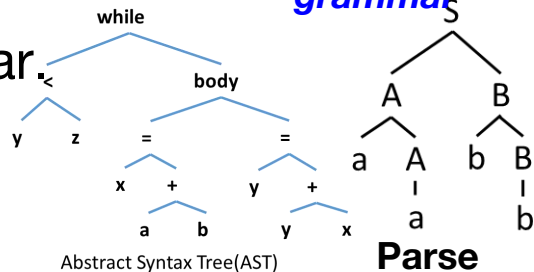


```
while (y<z){  
  int x = a +  
  b;  
  y += x; }
```

tokens

(keyword, **while**) (id, b)
(id, y) (sym, ;)
(sym, <) (id, y)
(id, z) (sym, +=)
(id, x) (id, x)
(id, a) (sym, +)
(sym, +)

Verifies tokens can be generated by the grammar



Abstract Syntax Tree(AST)

Parse Tree

Final Review—Syntax Analysis



- Context-free grammar

- sufficient ability to describe the most programming language.

- Derivation [应用产生式规则, 右部替换左部]: $\overset{*}{\Rightarrow}$, $\overset{+}{\Rightarrow}$, $\Rightarrow$;

- leftmost/rightmost.

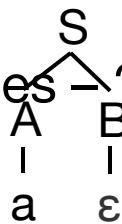
- Sentential form: $S \Rightarrow \alpha$; Sentence: α w/o V_N .

- The language generated by a grammar is its set of sentences [文法产生的句子全体].

- Derivations can be summarized as a parse tree [语法分析树] .

- ? Provide the derivation and draw parse tree.
- ? The internal nodes – non-terminals, The leaf nodes – terminals. ϵ is V_T ?

- yields [产出] or frontier [边缘] [从左至右排列符号构成句型]
e.g. $S \Rightarrow AB$; $A \Rightarrow a$; $B \Rightarrow b$ Sentence: a



• Ambiguity

- ? Determine a grammar is ambiguous.[某个句子对应两棵不同的语法树/最左/最右推导]
- ? Ambiguity of language **v.s.** Ambiguity of Grammar.
- Remove Ambiguity at the grammar level.
 - On the basis of **Precedence & Associativity**.
 - 分析二义性来源，同种符号间考虑结合性，不同符号考虑优先级。
 - 结合性: 左结合,左递归....
 - 优先级: 引入新的非终结符，往往伴随结合性带来的二义...



• Ambiguity

- ? Determine a grammar is ambiguous.[某个句子对应两棵不同的语法树/最左/最右推导]
- ? Ambiguity of language **v.s.** Ambiguity of Grammar.
- ? Remove Ambiguity at the grammar level.
 - On the basis of **Precedence & Associativity**.
 - 分析二义性来源，同种符号间考虑结合性，不同符号考虑优先级。
 - 结合性: 左结合,左递归....
 - 优先级: 引入新的非终结符，往往伴随结合性带来的二义...



 中山大學
 SUN YAT-SEN UNIVERSITY



Final Review—Syntax Analysis



- **Top-Down parsing:** find **leftmost** derivation, key problem: choose production.
- **Recursive-descent parsing:** a set of procedures, one for each non-terminal. Try to solve problem *in any possible ways* and cause **backtracking**.
- **Predictive Parsing:** looking ahead, and restrictions on the grammar to avoid backtracking. [LL/LL(1)]
- Two important problems and corresponding solution:
 - ? left recursion & remove left recursion. (judge & remove, 直接&间接)
 - ? common prefix & left factoring.
- ? How to determine whether a grammar *is suitable for **top-down parsing***?



RDP can handle common prefix (although expensive), but not left recursion

Final Review—Syntax Analysis



- ? How to determine LL(1) grammar.

$S \rightarrow BC$

◆ $A \rightarrow \alpha|\beta$: (LL(1) Parser Table, 根据问法)

◆ $FIRST(\alpha) \cap FIRST(\beta) = \phi$

◆ If $\epsilon \in FIRST(\beta)$, $FIRST(\alpha) \cap FOLLOW(A) = \phi$

- ? Compute FIRST & FOLLOW.

- 遍历产生式, 运用规则, 直到所有集合不再变化为止。
- 单个符号、串的FIRST计算都要掌握, FIRST里可能含空串。
- 先算FIRST再算FOLLOW, 因为FOLLOW计算要用FIRST信息, FOLLOW里没有空串, 不要忘记endmarker \$。

FIRST(B) add to FIRST(S)
 $\epsilon \in FIRST(B)$? FIRST(C) add to FIRST(S)
 $\epsilon \in FIRST(C)$? Place \$ in FOLLOW(S)
 FIRST(C) except ϵ add to FOLLOW(B)
 FOLLOW(S) add to FOLLOW(C)
 $\epsilon \in FIRST(C)$? FOLLOW(S) add to FOLLOW(B)

- ? LL(k) grammar. using k input symbols of lookahead at each step to make decisions.

LL(0) grammar—no lookahead, only one rule per non-terminal, only one string.

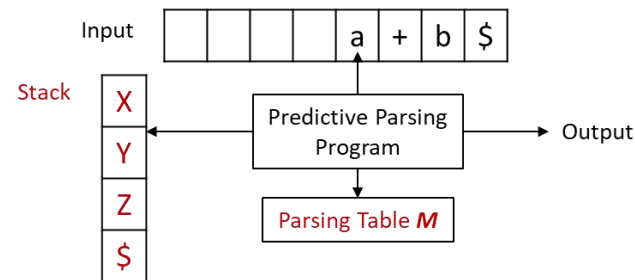
Can you write an LL(0) example?



Final Review—Syntax Analysis



- Non-recursive LL(1) Parser—Input buffer, Stack, Parsing Table $M[A,a]$, Program.
- ? Construct LL(1) Parse Table.
 - Use FIRST and FOLLOW sets.
 - $E \rightarrow TE'$: $\text{FIRST}(TE')$; $E' \rightarrow \epsilon$: $\text{FOLLOW}(E')$



- Remember Structure: Row & Column Number

E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

- 给定文法消左递归、提取左公因子,尝试变为LL(1);
- 根据FIRST&FOLLOW的计算结果构建LL(1)分析表,判断改写后的文法。
- 根据分析表分析一些输入。

Use LL(1) Parse Table.

步骤	符号栈	输入串	动作 (Derive/Match/Accept)	输出
0	S \$	a+b \$	Derive	$S \rightarrow AS'$
1	AS' \$	a+b \$	...	...

(注意: 符号栈左侧是栈顶)



Final Review—Syntax Analysis

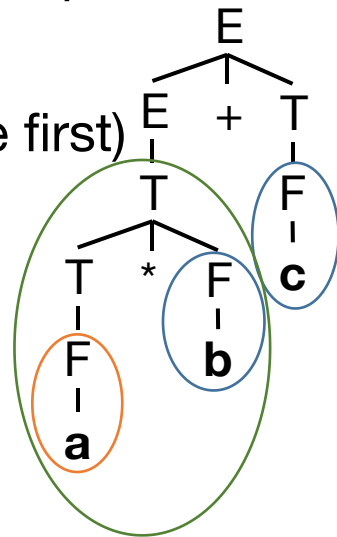


- **Bottom-Up parsing:** inverse process of rightmost derivation.
- **Shift-Reduce** parsing: (1) when to shift or reduce? (2) which production to apply?

- **Handle** ~~Phrase~~ ~~Simple Phrase~~ (usually draw; parse tree first)

短语：一个句型的语法树中任一子树的所有叶结点所组成的符号串；
直接短语：只有父子两代的子树形成的短语；

句柄：语法树中最左那棵只有父子两代的子树形成的短语；



- In practice, we should find handle before obtaining parse tree.
- Handle always occurs at stack top, no handle – shift, handle – reduce.

- **? Viable prefix:** $S \Rightarrow aEb \Rightarrow aaEb$, the handle is **aE**.

Viable Prefix : **a**, **aa**, **aaE**.

- Phrase: **a*b+c**, **a*b**, **a**, **b**, **c**
- Simple Phrase: **a**, **b**, **c**
- Handle: **a**

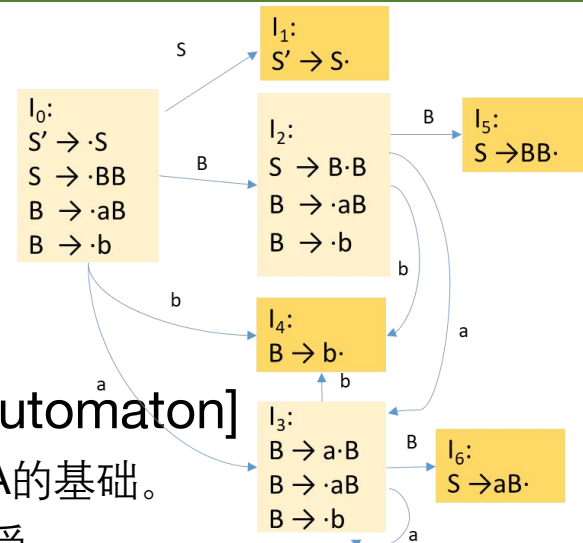


Final Review—Syntax Analysis



- Ensure the stack always contains viable prefixes.

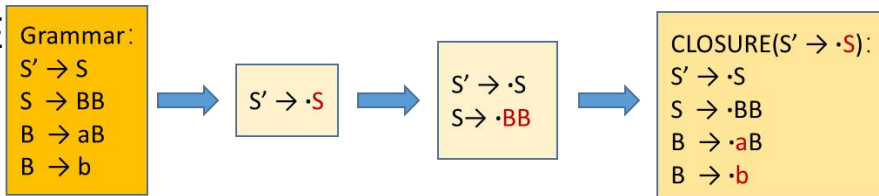
- LR family parsers. [many type of bottom-up parsing: OPP....]
- LR(k) is more complex and powerful than LL(k).
- LR(0), SLR(1), LR(1), LALR.



- ? Construct DFA recognizing viable prefixes. [LR(0) Automaton]**

- Item[项], 项集, 规范LR(0)项集族的一组项集提供了构建一个DFA的基础。
- Augmented Grammar: 告诉语法分析器何时停止语法分析并接受。

- CLOSURE



增广文法，构造初始状态，使用GOTO添加状态，重复操作，注意每个状态需要CLOSURE，同时注意已存在的相同状态。

- GOTO(I,X): I中所有形如 $[A \rightarrow \alpha \cdot X \beta]$ 的项所对应的 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包。

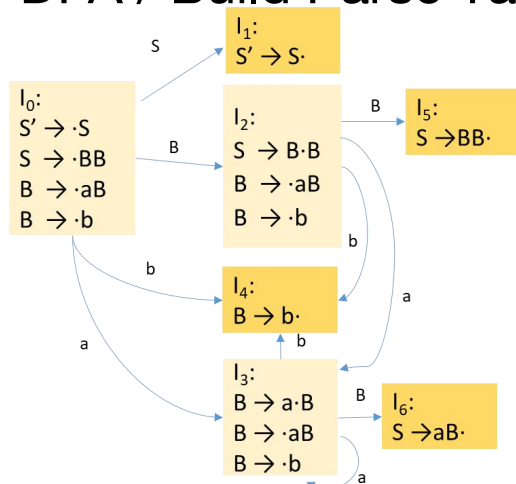


Final Review—Syntax Analysis



- ? Usage of DFA / Build Parse Table from DFA

- (0) $S' \rightarrow S$
 (1) $S \rightarrow BB$
 (2) $B \rightarrow aB$
 (3) $B \rightarrow b$



State	ACTION			GOTO	
	a	b	\$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

“together”

自己画表都
使用这种形
式



- ? Use LR Parse Table to parse.

步骤	栈		输入串	ACTION	GOTO
	状态	符号			
0	0	\$	ababa \$	s1	
1	01	\$a	baba\$	s2	
2	012	\$ab	aba\$	s1	
3	0121	\$aba	ba\$	s2	
4	01212	\$abab	a\$	s1	

State	ACTION		
	a	b	\$
0	s	s	
1			acc
2	s	s	
3	s	s	
4	r3	r3	r3
5	r1	r1	r1
6	r2	r2	r2

State	GOTO			
	a	b	S	B
0	3	4	1	2
1				
2	3	4		5
3	3	4		6
4				
5				
6				

“separately”

Remember

r

Structure:

Column
Number.



Final Review—Syntax Analysis



- ? Focus on **Conflicts**: Shift-Reduce & Reduce-Reduce (
- LR(0): 项目形式完整就归约($A \rightarrow \alpha \cdot$);
- ? SLR(1): Reduce only if the next input token is a member of the FOLLOW set.
 - $\text{FOLLOW}(T) \cap \text{FOLLOW}(V) \cap \{ '[' \} = ?$;
 - 与LR(0)的DFA相同，构建分析表时添加归约动作时不要忘记考虑FOLLOW信息;
- SLR(1) Limitation: 提供的FOLLOW信息太泛;
 - 在某个规范句型跟在 $\text{FOLLOW}(T)$ 后面的终结符，但是它并不能精确地指示在特定点的推导中， T 之后是哪些符号。
- ? LR(1): 通过分裂状态，对项进行精化，指明在哪些输入符号时才可以归约。
 - $A \rightarrow X_1 \dots X_j \cdot$, a [LR(0) item + lookahead terminals]
 - 移进项目的lookahead没有任何作用，仅用作传递；对归约项目起作用。

I_1 :
 $T \rightarrow i \cdot$
 $T \rightarrow$
 $i \cdot [E]$
 $V \rightarrow j$

Final Review—Syntax Analysis



- ? Construct LR(1) DFA Closure(): Dragon Book 2nd edition Section 4.7.2 Figure 4.19
 - ? Closure(): Given each item $[A \rightarrow u \cdot Bv, a]$ in I , for each production rule $B \rightarrow w$ in G' , add $[B \rightarrow \cdot w, b]$ to I :
 - $b \in \text{First}(va)$ && $[B \rightarrow \cdot w, b]$ is not in I ; v can be nullable.
 - ? Goto(I, X): For item $[A \rightarrow u \cdot Xv, a]$ in I , $\text{Goto}(I, X) = \text{Closure}([A \rightarrow uX \cdot v, a])$

same with LR(0)
propagate
lookahead
 - ? Construct LR(1) Parse Table.
 - Conflicts in LR(1): (1) $[A \rightarrow u \cdot xv, a] [B \rightarrow v \cdot, x]$
 (2) $[A \rightarrow u \cdot, a] [B \rightarrow v \cdot, a]$
- ? LALR (lookahead LR) – compromise LR(1) and SLR(1)[折衷]

Existing merge conflict

c	d
$B \rightarrow e \cdot,$	$B \rightarrow e \cdot,$
d	c

↓

$C \rightarrow e \cdot, c/d$
$B \rightarrow e \cdot, d/c$

 - The core of each item is identical, and they differ only in their lookahead
 - **Cannot** introduce new shift-reduce (s-r) conflicts & **Can** introduce **reduce-reduce** (r-r) conflicts
 - **Detection of errors** may be delayed[推迟错误识别], more reductions.

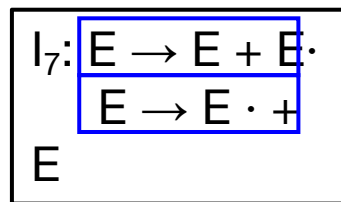
Final Review—Syntax Analysis



- ? Relationship of Grammars. [一些解题技巧]
- ? **Resolving ambiguities at the parsing table**

- 二义文法则一定不是LL和LR的,无法使用它们来消歧。
- 借助算符的优先级和结合性来消除二义性。

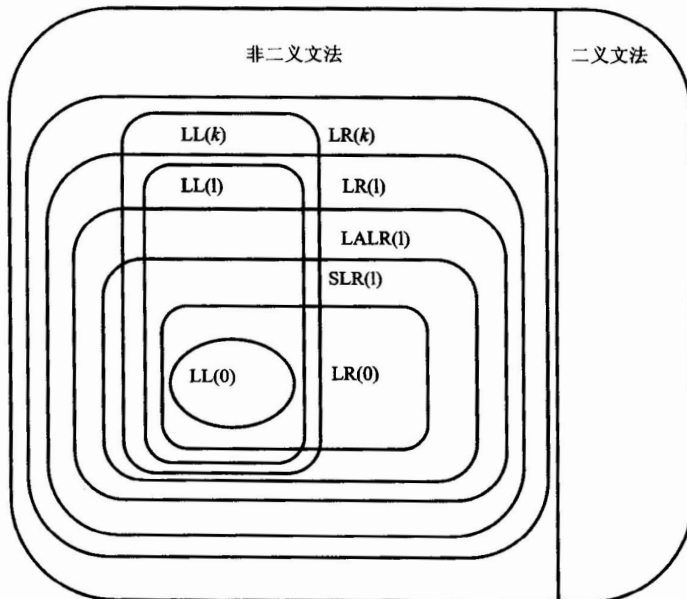
$E \rightarrow E+T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$



$E \rightarrow E \cdot *$

E运算符+的结合性可以决定如何解决这个冲突。

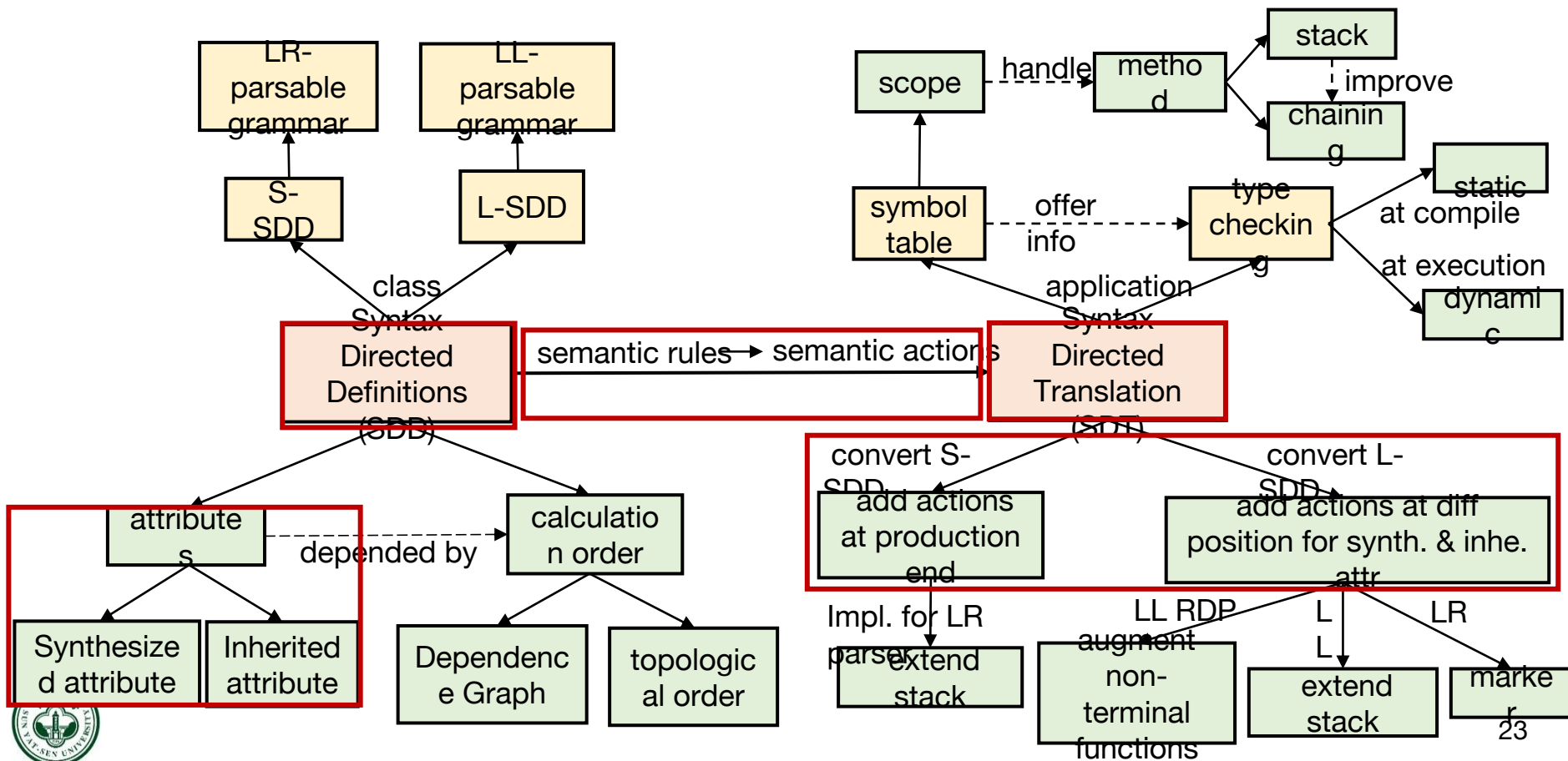
	Stack	Input	ACTION	GOTO
0	\$	id+id+id\$	s3	
03	\$id	+id+id\$	r4($E \rightarrow id$)	
0	\$	+id+id\$		1
01	\$E	+id+id\$	s4	
014	\$E+	id+id\$	s3	
0143	\$E+id	+id\$	r4($E \rightarrow id$)	
014	\$E+	+id\$		7
0147	\$E+E	+id\$	r1 or s4?	



[Asg2 Q3]



Final Review – Semantic Analysis

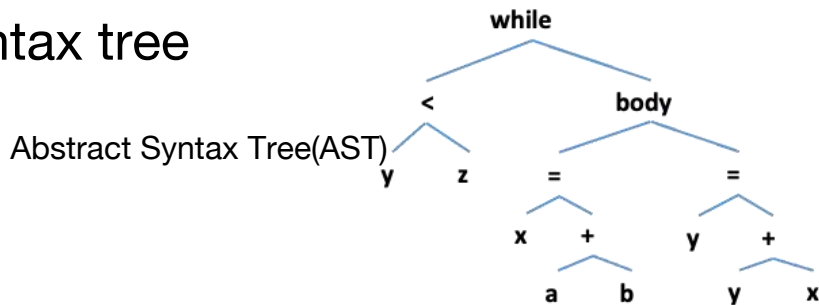


Final Review—Semantic Analysis



- After Syntax Analysis, we obtain the syntax tree

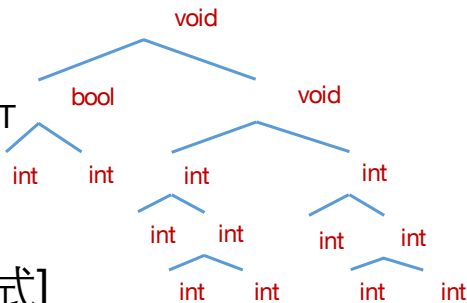
- The structure of the tokens.
- The tree is then passed to the next stage.



- Next, the semantic analysis is conducted

- Associates **identifiers** with **objects** they refer to.
- Checks whether identifiers are used correctly.

Annotated AST/Decorated AST
[带标注的抽象语法树]



- Syntax: describes the **proper form** of the programs [仅形式]
- Semantics: defines **the meaning of programs** (i.e., what each program does when it executes) [到意义]



Final Review—Semantic Analysis



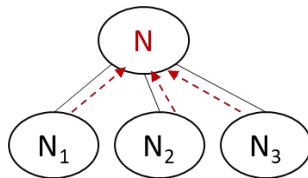
- SDD has two types of attributes [文法符号的两种属性]

- ◆ **Synthesized attribute** [综合属性]

- ◆ **Inherited attribute** [继承属性]

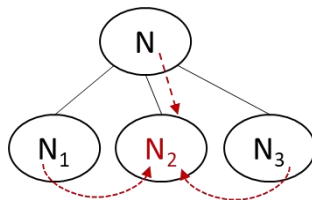
- A synthesized attribute of node N is defined only **by attributes of N's children and N itself** [子节点或自身]

- $N \rightarrow N_1 N_2 N_3 \dots$



- An inherited attribute of node N is defined only **by attribute values at N's parent, N itself, and N's siblings** [父节点、自身或兄弟节点]

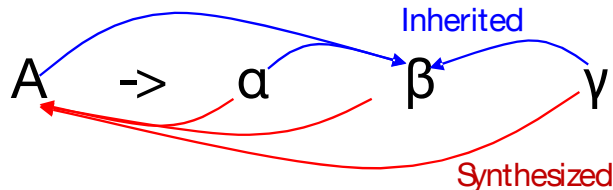
- $N \rightarrow N_1 N_2 N_3 \dots$



Final Review—Semantic Analysis



- **Attribute dependencies** in a production rule [产生式中的属性依赖]



- Given $b = f(A.attrs, \alpha.attrs, \beta.attrs, \gamma.attrs)$
 - ◆ ? Is b synthesized or inherited
- b is either an attribute in LHS (an attribute of A)
 - ◆ In which case b is a **synthesized** attribute
 - ◆ From A 's perspective α, β, γ are children
- Or, b is an attribute in RHS (e.g., of β)
 - ◆ In which case b is an **inherited** attribute
 - ◆ From β 's perspective A, α, γ are parent or siblings



Final Review—Semantic Analysis



- An SDD is **S-attributed** if every attribute is synthesized [只具有综合属性]
 - ◆ Can evaluate its attributes in **any bottom-up order** [自底向上的顺序计算属性值]
 - ◆ Can be implemented during bottom-up parsing [与自底向上的语法分析同时进行]
- An SDD is **L-attributed** (L-SDD) if
 - ◆ Each attribute can be either **synthesized, or inherited** with the rules that suppose $A \rightarrow X_1 X_2 \dots X_{i-1} X_i X_{i+1} \dots$, the inherited attribute $X_i.a$ only depends on:
 1. **Inherited** attributes associated with A [A的继承属性, Why?]
 2. Either **syn or inh** attributes of $X_1, \dots, X_{i-1}$ located to the **left** of X_i [X_i 的之前符号的综合/继承属性]
 3. Either **syn or inh** attributes of X_i itself, but **no cycles** formed by the attributes of this X_i [X_i 的综合/继承属性 (不能出现环形依赖)]

S-SDD中不会出
现继承属性

L-SDD中, RHS继承属性不会依赖LHS综合
属性

L-SDD中, RHS不会出现从右到左的依赖关
系



Final Review—Semantic Analysis



- SDD is S-attributed, the underlying grammar is LR-parsable
- SDD is L-attributed, the underlying grammar is LL-parsable
 - But L-SDD can be parsed by a LR parser by modifying the grammar.
- Implementation of S-SDD
 - Place each action at the end of the production [将每个语义动作都放在产生式的最后]
 - SDTs with all actions at the right ends of the production are **Postfix SDT** [后缀/尾部SDT]

S-SDD

Productions	Semantic rules
(1) $L \rightarrow E$	$print(E.val)$
(2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
(3) $E \rightarrow T$	$E.val = T.val$
(4) $T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
(5) $T \rightarrow F$	$T.val = F.val$
(6) $F \rightarrow (E)$	$F.val = E.val$
(7) $F \rightarrow digit$	$F.val = digit.lexval$

SDT

CFG with actions
(1) $L \rightarrow E \{print(E.val)\}$
(2) $E \rightarrow E_1 + T \{E.val = E_1.val + T.val\}$
(3) $E \rightarrow T \{E.val = T.val\}$
(4) $T \rightarrow T_1 * F \{T.val = T_1.val * F.val\}$
(5) $T \rightarrow F \{T.val = F.val\}$
(6) $F \rightarrow (E) \{F.val = E.val\}$
(7) $F \rightarrow digit \{F.val = digit.lexval\}$



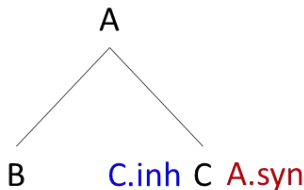
Final Review—Semantic Analysis



- Implementation of L-SDD
 - Embed the action that computes the **inherited attributes** for a nonterminal **A** immediately **before the occurrence of A** in the production body
 - Place the actions of **synthesized attribute** for the LHS **at the end of the production body**

- **A $\rightarrow$ BC**

- ◆ C的**继承**属性: 在**C之前**
- ◆ A的**综合**属性: 在**RHS末尾**



- Semantic translation during **parsing**
 - ◆ An **LL** recursive-descent parser[递归下降的LL分析]
 - **Augment non-terminal functions** to both **parsing** and **attributes handling**
 - ◆ An **LL** predictive parser[非递归的LL预测分析]
 - **Extend the parse stack** to **hold actions** and certain **data items** needed for attribute evaluation
 - ◆ An **LR** parser[LR分析]
 - **Rewrite grammars** to involve **marker**



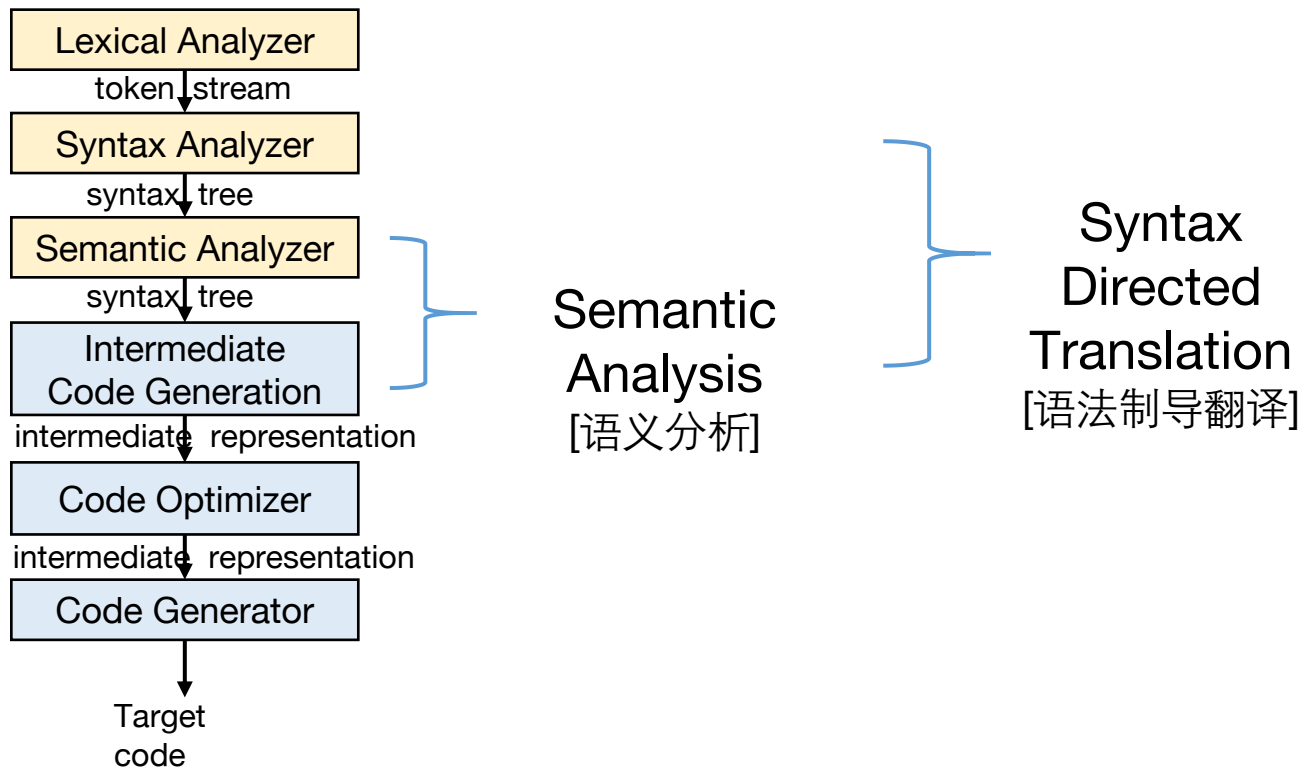
Final Review—Semantic Analysis



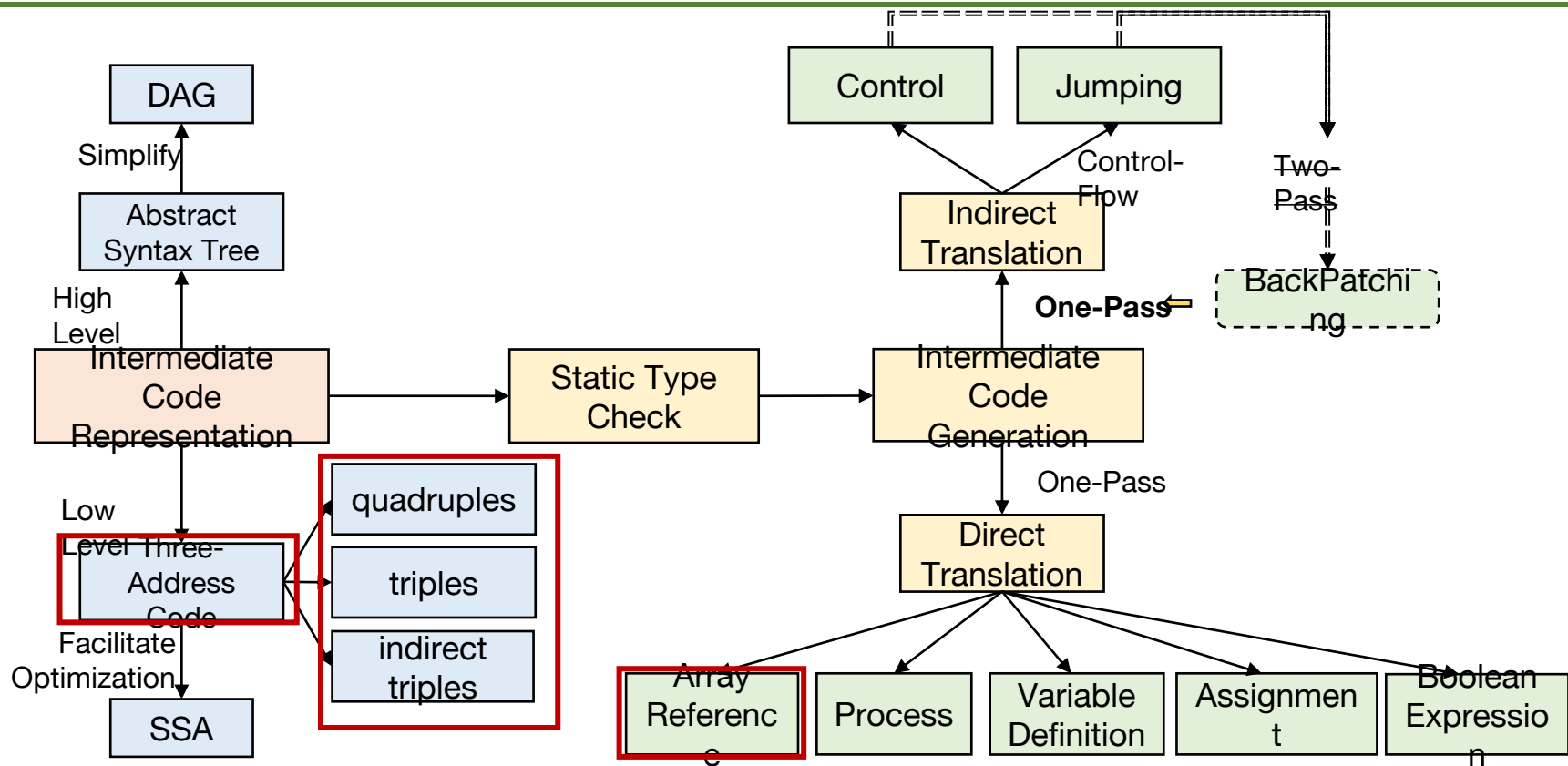
- ? Tell whether a given attribute is L or S
 - [Dragon Book 2nd edition]
- ? Given an SDT, compute the result
- ? Given a grammar, write semantic rules to address a problem
- ? Given a SDT and an input sentence, construct the Annotated Parse Tree
 - [Dragon Book 2nd edition, Fig 5.3-5.5, Example 5.1.1]



- **Syntax Directed Translation**[语法制导翻译]



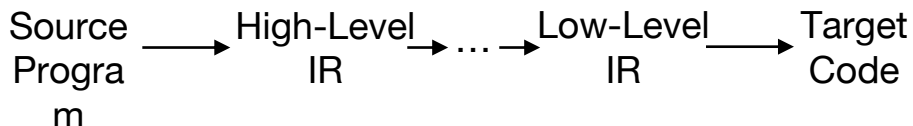
Final Review—Intermediate Code



Final Review—Intermediate Code



• Intermediate Representation [中间表示]



• High-level IR are close to the source code [接近源语言]

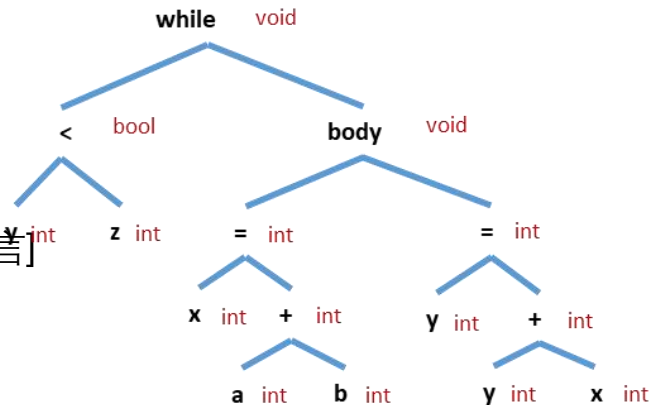
- ◆ Example: **Parse Tree**, **Abstract Syntax Tree** [抽象语法树]
- ◆ Language **dependent** (for each language)
- ◆ Purpose: semantic analysis of program

• Low-level IR are close to assembly [接近汇编]

- ◆ Example: **three address code** (TAC) [三地址码]
- ◆ Essentially an instruction set [指令集] for an abstract machine

• Machine-Level IR [机器层级]

- ◆ Example: x86 IR, ARM IR, ...
- ◆ Actual instructions for a concrete machine ISA [指令集架构]
- ◆ Machine **dependent** (a machine-level IR for each ISA)



Annotated AST/Decorated AST
[带标注的抽象语法树]

```
goto L1
L2:
    t1 := a + b
    x := t1
    t2 := y + x
    y := t2
L1:
    if y < z goto L2
```



- **Three-Address Code**[三地址代码]
 - Generic form is $X = Y \text{ op } Z$ [最多3个操作数]
 - where X, Y, Z can be variables, constants, or compiler-generated temporaries holding intermediate values.
- Three representations. (and more)
 - ◆ **quadruples**. [四元式]
 - ◆ **triples**. [三元式]
 - ◆ **indirect triples**. [间接三元式]
- Example: $a * b + a * b$ is translated to
 - $t_1 = a * b$
 - $t_2 = a * b$
 - $t_3 = t_1 + t_2$
 - t_1, t_2, t_3 are temporary variables

Final Review—Intermediate Code



- **Quadruples.** [四元式]

- four fields: op , arg_1 , arg_2 , $result$.
- $x = y + z \Rightarrow (+, y, z, x)$

step	instruction
0	(0)
1	(1)
2	(4)
3	(5)

- **Triples.** [三元式]

- three fields: op , arg_1 , arg_2 .
- $x = y + z \Rightarrow (+, y, z)$

index	op	arg ₁	arg ₂
(0)	minus	c	
(1)	*	b	(0)
(2)		empty	
(3)		empty	
(4)	+	(1)	(1)
(5)	=	a	(4)

- **Indirect triples.** [间接三元式]

- Triples are stored in a 'database'

- **Single Static Assignment**

- Every variable is assigned exactly once statically [仅一次]
- ? Write TAC for a given expression in the specified form

Assignment statement:

$a = b + (-c)$

Three-address code:

$t_1 = \text{minus } c$

$t_2 = b + t_1$

$a = t_2$



Final Review—Intermediate Code



- We will use the **syntax-directed formalisms** to specify translation
 - ◆ Variable definitions[变量定义] -> Recall semantic analysis & symbol table
 - ◆ Assignment[赋值]
 - ◆ **Array references**[数组引用]
 - ◆ Find the address of an element
 - ◆ **Boolean expressions**[布尔表达式]
 - ◆ $E \rightarrow \text{if } (B) \text{ then } X \text{ else } Y$, **E.next**, **B.false**, **B.true**
 - ◆ Control-flow statements[控制流语句]
- To generate three-address codes (TACs)
 - ◆ Lay out variables in memory (**location** and **width**)
 - ◆ Generate TAC for any sub-expressions or sub-statements
 - ◆ Using the result, generate TAC for the overall expression



Final Review—Intermediate Code



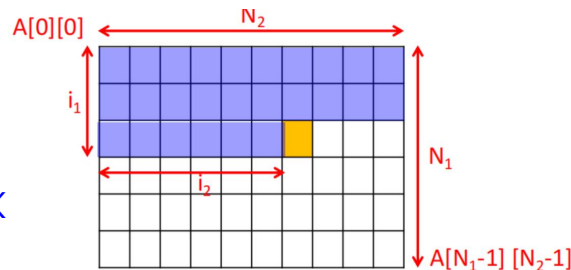
- 1D array: `int A[N]`

- **base**: address of the first element
- **width**: width of each element
- $\text{addr}(A[i]) = \text{base} + i \times \text{width}$



- 2D array: `int A[N1][N2]` (row major)

- $\text{addr}(A[i_1, i_2]) = \text{base} + (i_1 \times \underline{N_2 \times \text{width}} + i_2 \times \underline{\text{width}})$



- k-dimensional array

- $\text{addr}(A[i_1][i_2] \dots [i_k]) = \text{base} + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$
- `Type(a) = array(3, array(5, array(8, int)))`
- $\text{addr}(a[i_1][i_2][i_3]) = \text{base} + i_1 \times w_1 + i_2 \times w_2 + i_3 \times w_3 = \text{base} + i_1 \times 160 + i_2 \times 32 + i_3 \times 4$

- ? Compute the address for a given array and indexes



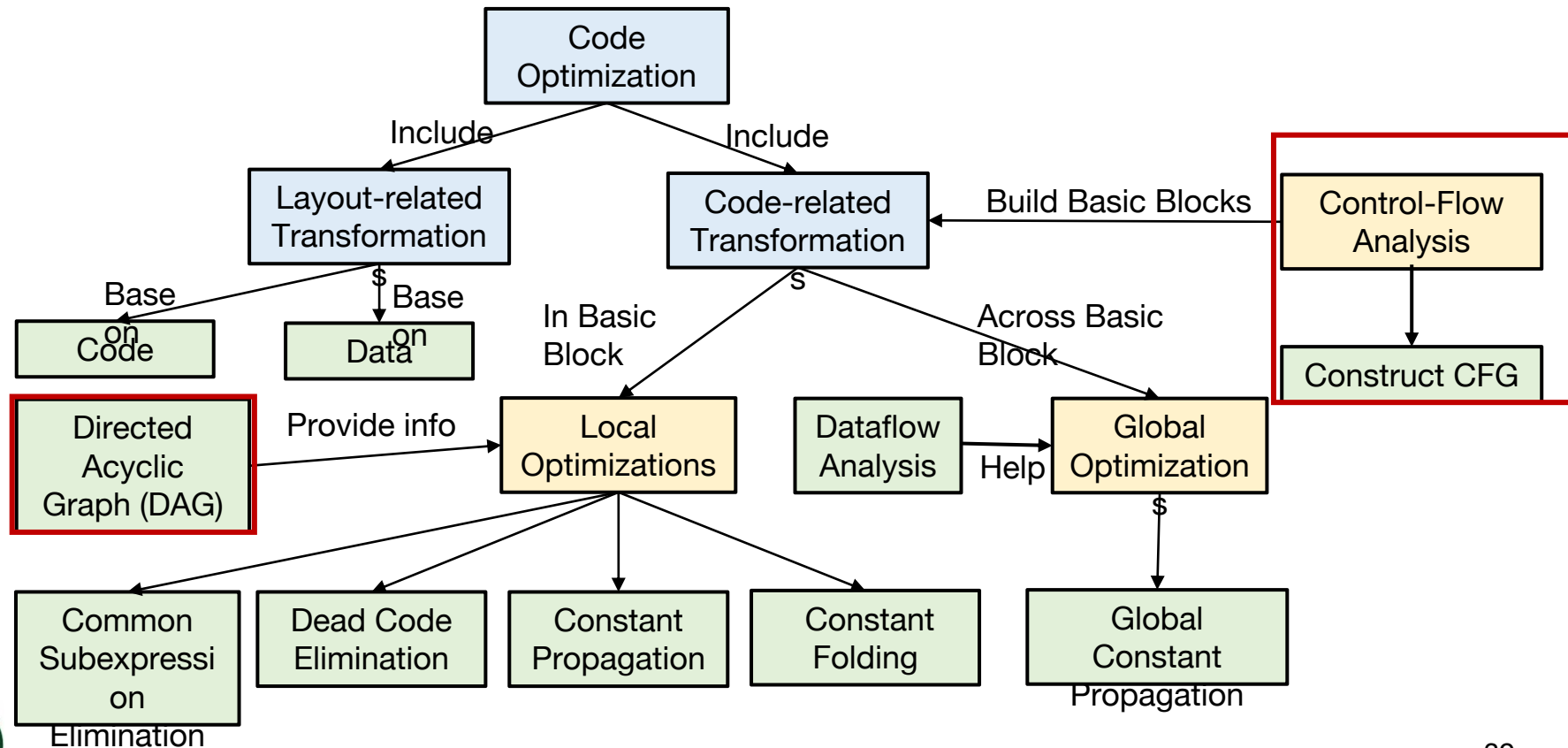
Final Review—Intermediate Code



- For control flow statements, code generation needs to specify the target address of jumps.
- However, we might not know the target address until that part is generated.
- **Backpatching**[回填]
 - Leave **holes** in IR in place of forward jump addresses
 - Record indices of jump instructions in **a hole list**
 - When target address of label for jump is eventually known, **backpatch holes** using the hole list for that particular label
 - ? 补全给的表达式的翻译结果



Final Review—Code Optimization



Final Review—Code Optimization

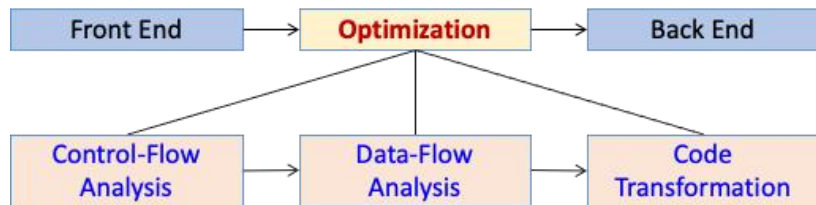


- Goal of Optimization:
 - ◆ Execution time / Memory usage / Energy consumption / Binary executable size...
 - ◆ Equivalence / Efficient / Cost-effectiveness
- **Layout-related** transformations[布局相关]
 - ◆ Optimizes where in **memory** code and data is placed
 - ◆ Goal: **maximize the spatial locality** [空间局部性].
 - ◆ code layout, declaration ordering, data structuring
- **Code-related** transformations[代码相关]
 - ◆ Optimizes what **code** is generated.
 - ◆ Goal: execute **least number of most costly instructions**.

```
goto L1
L2:
    t1 := a + b
    x := t1
    t2 := y + x
    y := t2
L1:
    if y < z goto
```



```
t1 = a + b
goto L1
L2:
    t2 := y + t1
    y := t2
L1:
    if y < z goto
```



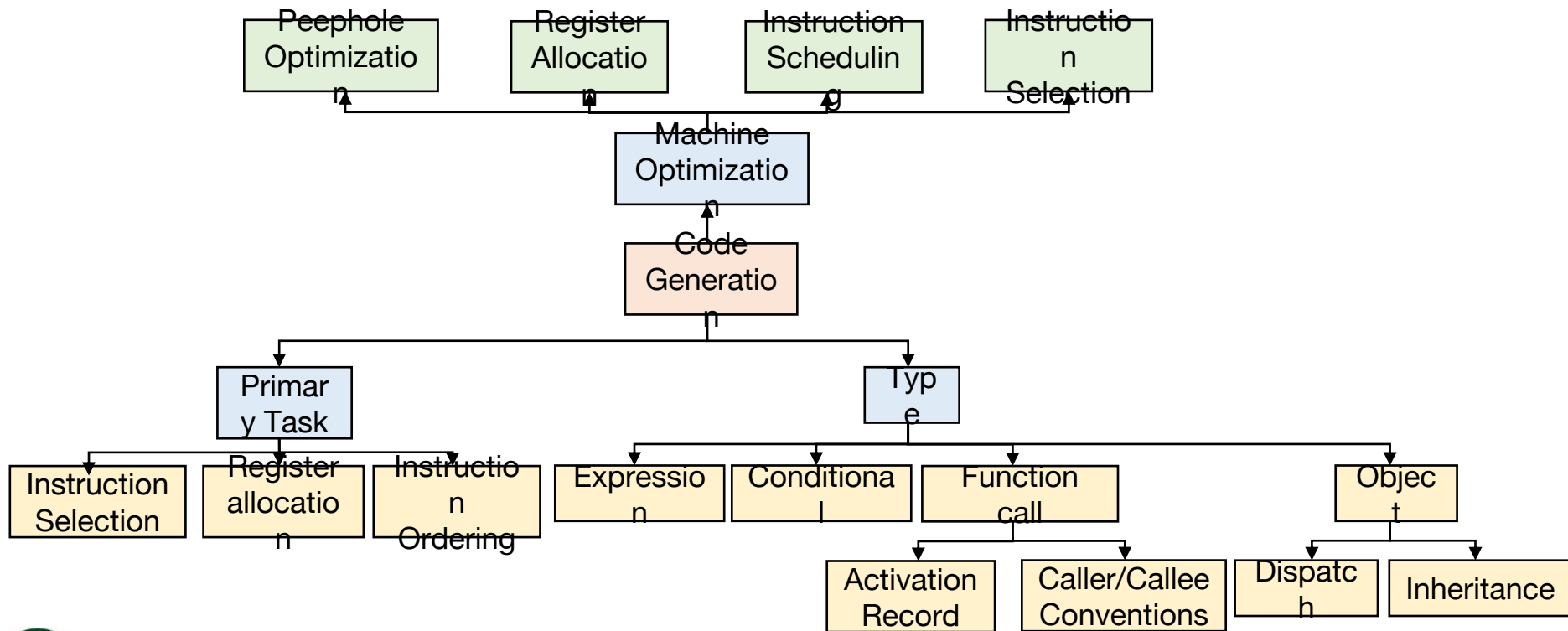
Final Review—Code Optimization



- Code-related optimization
 - Basic blocks / Control-flow graph (CFG) / Directed acyclic graphs (DAG)
 - ◆ Local optimization – within one basic block
 - CSE / DCE / Algebraic identities / Constant folding and propagation
 - ◆ Global Optimization – cross multiple basic blocks, functions, etc
 - Constant propagation
- ? Given a code segment
 - specify the basic blocks
 - Construct the CFG
- ? Given a basic block
 - Specify the liveness of variables
 - construct its DAG and optimize



Final Review—Target Code



Final Review—Target Code



- Target code generation
 - ◆ **Transform** the optimized **intermediate code** into **target code**.
- Three primary tasks:
 - ◆ **Instruction selection**[指令选取]
 - Choose **appropriate** target-machine **instructions** to implement the IR statements
 - The cost of instructions[指令开销]
 - ◆ **Register allocation and assignment**[寄存器分配]
 - decide **what values** to keep in **which register**
 - Graph coloring & register spilling
 - ◆ **Instruction ordering**[指令排序]
 - Decide in **what order** to schedule the execution **of instructions**
 - Peephole optimization



Final Review—Target Code



- **Instruction cost** = 1 + cost (source-mode) + cost (destination-mode)

Mode	Form	Address	Added Cost
Absolute	M	M	1
Register	R	R	0
Indexed	<i>c(R)</i>	<i>c+contents(R)</i>	1
Indirect register	*R	<i>contents(R)</i>	0
Indirect indexed	*c(R)	<i>contents(c+contents(R))</i>	1
Literal	#c	N/A	1

- Examples

Instruction	Operation	Cost
MOV R0, R1	Store <i>content(R0)</i> into register R1	1
MOV R0, M	Store <i>content(R0)</i> into memory location M	2
MOV M, R0	Store <i>content(M)</i> into register R0	2
MOV 4(R0), M	Store <i>contents(4+contents(R0))</i> into M	3
MOV *4(R0), M	Store <i>contents(contents(4+contents(R0)))</i> into M	3
MOV #1, R0	Store 1 into R0	2
ADD 4(R0), *12(R1)	Add <i>contents(4+contents(R0))</i> to <i>contents(12+contents(R1))</i>	3

- ? Given a code segment, compute its cost



Final Review—Target Code



- **Code**

- the size of the generated target code is fixed at compile time

- **Global/static**

- the size of some program data objects, e.g., global constants, are known at compile time

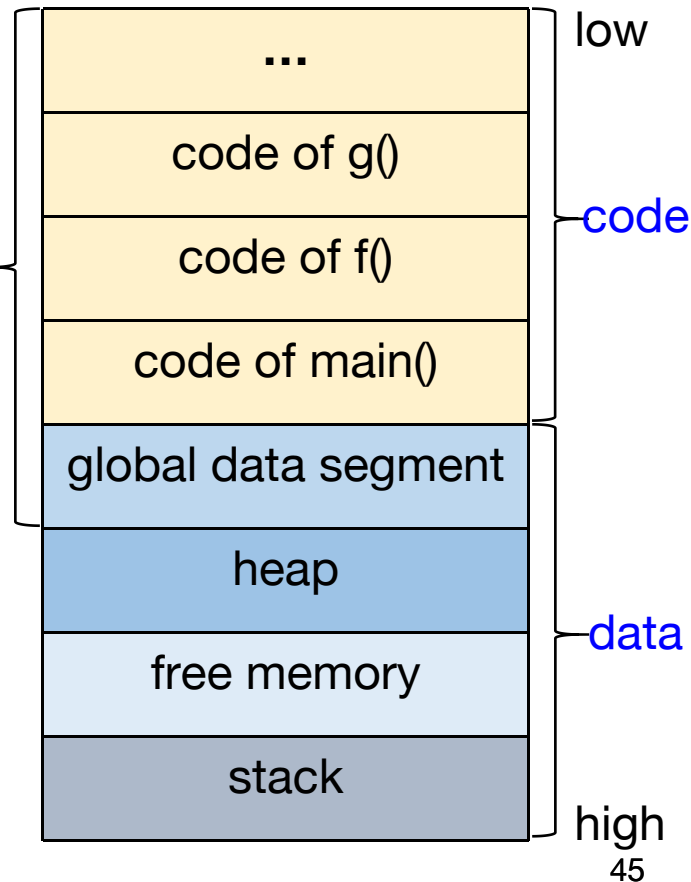
- **Stack**

- store dynamic data structures

- **Heap**

- manage long-lived data
- Garbage Collector – [Referencing Counting](#)
[引用计数]
 - Count the number of references to an object
 - Collect when the count equals to 0

executable
image



Concluding Remarks



- The journey is now coming to its end (almost)
 - It is quite a long journey
 - Sometimes intensive (5 Quiz, 5 assignments, and 4 labs)
- So, what is exactly **compilation**?
 - To systematically transform **human readable** programming languages to **executable** code by a computer
 - To improve the code performance
- More importantly, it is about
 - The fundamental **design philosophy** in computer science
 - The ability of **designing systems** for problems addressing
- **Thank you all for the attendance!!**

